Java(sem)-5

Q1: What are the five keywords used in Java exception handling? Explain with suitable code snippets

Definition

Exception handling in Java is a mechanism used to handle **runtime errors (exceptions)** so that the normal flow of the program is not interrupted.

Keywords Used

The five keywords used in Java exception handling are:

- try
- catch
- finally
- throw
- throws

Key Points

- Prevents program crash
- Maintains normal execution flow
- Separates error-handling code
- Improves reliability and readability

Explanation with Syntax & Code

1) try and catch

The **try block** contains code that may cause an exception, and the **catch block** handles the exception.

Syntax

```
try {
   // risky code
} catch (ExceptionType e) {
   // handling code
}
```

Example

```
public class TryCatchDemo {
   public static void main(String[] args) {
      try {
         int a = 10 / 0;
```

```
        } catch (ArithmeticException e) {
            System.out.println("Exception handled: Division by zero");
        }
    }
}
```

2) finally; The **finally block** is always executed, whether an exception occurs or not

Syntax

```
try {
    // code
} catch (Exception e) {
    // handling
} finally {
    // always executed
}
```

Example

```
public class FinallyDemo {
    public static void main(String[] args) {
        try {
            int a = 5 / 0;
        } catch (Exception e) {
            System.out.println("Exception occurred");
        } finally {
            System.out.println("Finally block executed");
        }
    }
}
```

---

3) throw

The **throw keyword** is used to explicitly throw an exception

Syntax

```
throw new ExceptionType("message");
```

Example

```
public class ThrowDemo {
    public static void main(String[] args) {
        int age = 15;

        if (age < 18) {
            throw new ArithmeticException("Not eligible to vote");
        }

        System.out.println("Eligible");
```

```
    }
}
```

---

4) throws

The **throws keyword** is used to declare exceptions that a method may throw.

Syntax

```
returnType methodName() throws ExceptionType {
    // code
}
```

Example

```
import java.io.*;

public class ThrowsDemo {
    static void readFile() throws IOException {
        FileReader fr = new FileReader("file.txt");
    }

    public static void main(String[] args) {
        try {
            readFile();
        } catch (IOException e) {
            System.out.println("File not found");
        }
    }
}
```

---

5. Output

```
Exception handled: Division by zero
Exception occurred
Finally block executed
Exception in thread "main" java.lang.ArithmeticException: Not eligible to vote
File not found
```

Exception Handling Flow in Java

```
        Start
          |
          v
       try block
          |
   -----------------
   |               |
No Exception   Exception Occurs
   |               |
   v               v
 Continue      catch block
 Execution         |
   |               v
   |           finally block
   |               |
   -------→ finally block
                    |
                    v
                  End
```

6. Conclusion

The five keywords—try, catch, finally, throw, and throws—form the core of Java exception handling, enabling programs to handle errors efficiently and continue execution smoothly.

2. Explain the concept of multiple catch clauses in Java and write a program to illustrate how different exceptions are handled using multiple catch blocks.

Definition

Multiple catch clauses in Java allow a single try block to be followed by **multiple catch blocks**, where each block handles a **different type of exception**.

Import Statements

import java.io.IOException; // for checked exception example

(Note: Many exceptions like ArithmeticException belong to java.lang and are imported automatically.)

Types of Exceptions in Java

1. Checked Exceptions

- Checked at **compile time**
- Must be handled using try-catch or throws
- Example: IOException

2. Unchecked Exceptions

- Occur at **runtime**
- Not mandatory to handle
- Example: ArithmeticException, ArrayIndexOutOfBoundsException

---

Syntax

```
try {
   // risky code
}
catch(ExceptionType1 e) {
   // handle exception 1
}
catch(ExceptionType2 e) {
   // handle exception 2
}
catch(Exception e) {
   // general handler
}
```

Explanation

- A try block may generate **multiple types of exceptions**
- Each catch block handles a **specific exception**
- Java checks catch blocks from **top to bottom**
- Only the **first matching catch block executes**

Properties

• Only **one catch block executes**
• **Order is important** (specific → general)
• Exception must be placed **last**
• Improves **clarity and control in error handling**
• Prevents abnormal program termination

Example Program

```
import java.io.IOException;

public class MultiCatchDemo {
   public static void main(String[] args) {

      try {
         int arr[] = {10, 20, 30};

         int a = 10 / 0;             // ArithmeticException
         System.out.println(arr[5]);  // ArrayIndexOutOfBoundsException

      }
      catch (ArithmeticException e) {
         System.out.println("Arithmetic Exception handled");
      }
      catch (ArrayIndexOutOfBoundsException e) {
         System.out.println("Array Index Exception handled");
      }
      catch (Exception e) {
         System.out.println("General Exception handled");
      }

      System.out.println("Program continues...");
   }
}
```

Output

Arithmetic Exception handled
Program continues...

Key Points

• Multiple catch blocks handle **different exceptions separately**
• Helps in **precise error handling**
• Improves program **robustness**
• Widely used in real-world applications

Conclusion

Multiple catch clauses allow efficient handling of different types of exceptions in a structured manner, improving program reliability and readability.

3. Write a Java program to explain User-Defined Exceptions

Definition

A **user-defined exception** (custom exception) is an exception created by the programmer by extending the Exception class. It is used to handle **application-specific errors**

Syntax

```
class MyException extends Exception {
   MyException(String msg) {
      super(msg);
   }
}
```

Explanation

To create and use a user-defined exception:

• Create a class that **extends Exception**
• Define a constructor to pass message
• Use throw to generate exception
• Handle using try-catch

Properties

- Created by extending **Exception class**
- Also called **custom exceptions**
- Used for **application-specific conditions**
- Can store custom error messages
- Must be handled using **try-catch or throws**
- Supports **reusability and clarity**
- Can override methods like toString()

Example Program

```
class InvalidAgeException extends Exception {

   InvalidAgeException(String msg) {
      super(msg);
   }
}

public class Test {
   public static void main(String[] args) {

      int age = 16;

      try {
         if (age < 18) {
            throw new InvalidAgeException("Not eligible to vote");
         } else {
            System.out.println("Eligible to vote");
         }
      }
      catch (InvalidAgeException e) {
         System.out.println(e.getMessage());
      }
   }
}
```

Output

Not eligible to vote

Conclusion

User-defined exceptions allow programmers to define and handle custom errors effectively, improving program readability and control.

4. What are unchecked exceptions in Java? Explain the commonly used built-in unchecked exceptions with examples.

Definition

Unchecked exceptions are exceptions that occur at **runtime** and are **not checked at compile time**. They are not mandatory to handle using try-catch or throws.

They are subclasses of **RuntimeException**.

Syntax (Example Handling)

```
try {
   // risky code
} catch (Exception e) {
   System.out.println(e);
}
```

Explanation

- Occur due to **programming errors** (logic mistakes)
- Not checked by compiler
- Can cause **program termination** if not handled
- Belong to java.lang package (auto-imported)

Common Built-in Unchecked Exceptions with Examples

1. ArithmeticException

Occurs when **invalid arithmetic operation** is performed (e.g., divide by zero)

```
int a = 10 / 0;
```

2. NullPointerException

Occurs when trying to use a **null object reference**

```
String s = null;
System.out.println(s.length());
```

3. ArrayIndexOutOfBoundsException

Occurs when accessing an **invalid index of array**

```
int arr[] = {1, 2, 3};
System.out.println(arr[5]);
```

4. NumberFormatException

Occurs when converting an **invalid string to number**

```
int x = Integer.parseInt("abc");
```

5. ClassCastException

Occurs when **incompatible type casting** is done

```
Object obj = "Hello";
Integer i = (Integer) obj;
```

Example Program

```
public class UncheckedDemo {
    public static void main(String[] args) {

        try {
            int a = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception handled");
        }

        try {
            String s = null;
            System.out.println(s.length());
        } catch (NullPointerException e) {
            System.out.println("Null Pointer Exception handled");
        }
    }
}
```

Output

```
Arithmetic Exception handled
Null Pointer Exception handled
```

Properties

• Occur at **runtime**
• Not checked by compiler
• Subclass of RuntimeException
• Not mandatory to handle
• Caused by **logical/programming errors**

Conclusion

Unchecked exceptions occur during program execution due to coding mistakes. Proper handling improves program stability and prevents unexpected crashes.

5. Write a Java Program to Solve Producer–Consumer problem using synchronization and Inter-Thread Communication

Definition

The **Producer–Consumer problem** is a classic synchronization problem where one thread (**producer**) generates data and another thread (**consumer**) uses that data. Proper coordination is required to avoid **race conditions** and ensure correct data sharing.

Concepts Used

• **Synchronization** → ensures only one thread accesses shared resource at a time
• **wait()** → makes thread wait until notified
• **notify()** → wakes up waiting thread
• **Shared resource** → common data between threads

Syntax (Inter-thread communication)

```
synchronized(obj) {
   while(condition) {
      obj.wait();
   }

   // perform operation
   obj.notify();
}
```

Explanation

• Producer adds data when buffer is empty
• Consumer removes data when buffer is full
• If buffer is full → producer waits
• If buffer is empty → consumer waits
• wait() and notify() coordinate execution

Program

```
class Shared {
    int data;
    boolean hasData = false;

    synchronized void produce(int value) {
        try {
            while (hasData) {
                wait();
            }

            data = value;
            System.out.println("Produced: " + data);
            hasData = true;

            notify();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    synchronized void consume() {
        try {
            while (!hasData) {
                wait();
            }

            System.out.println("Consumed: " + data);
            hasData = false;

            notify();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

class Producer extends Thread {
    Shared obj;

    Producer(Shared obj) {
```

```
        this.obj = obj;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            obj.produce(i);
        }
    }
}

class Consumer extends Thread {
    Shared obj;

    Consumer(Shared obj) {
        this.obj = obj;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            obj.consume();
        }
    }
}

public class Main {
    public static void main(String[] args) {

        Shared obj = new Shared();

        Producer p = new Producer(obj);
        Consumer c = new Consumer(obj);

        p.start();
        c.start();
    }
}
```

Output (Sample)

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
```

Produced: 5
Consumed: 5

Key Points / Properties

• Uses **synchronization** to avoid race conditions
• wait() and notify() enable communication
• Ensures **data consistency**
• Avoids **overproduction and under-consumption**
• Demonstrates **multithreading coordination**

Conclusion

The Producer–Consumer problem is solved using synchronization and inter-thread communication in Java. It ensures proper coordination between threads, maintaining data integrity and efficient execution.

6. Illustrate JDBC architecture with a neat sketch

Definition

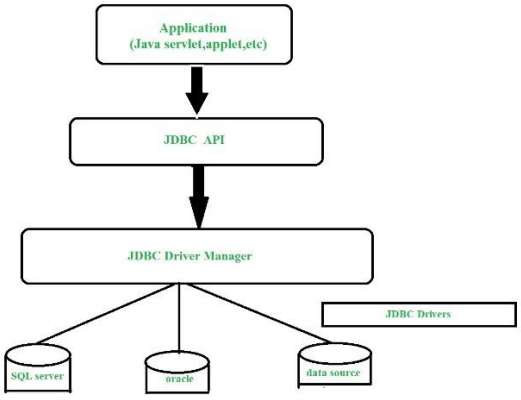
JDBC (Java Database Connectivity) is a standard Java API that allows Java applications to **connect and interact with databases**. It provides a common interface to communicate with different database systems.

Explanation

The JDBC architecture consists of multiple layers that help in establishing communication between a Java application and a database.

From your diagram, the architecture can be understood as follows:

1. **JDBC Architecture (Neat Sketch Explanation)**



- **Application Layer**
  This is the Java program (Servlet, Applet, or standalone application) that needs to interact with the database.

- **JDBC API**
  It provides classes and interfaces (like Connection, Statement, ResultSet) to send SQL queries and process results.
- **Driver Manager**
  It manages different database drivers and selects the appropriate driver based on the database.
- **JDBC Drivers**
  These are database-specific drivers that convert Java calls into database-specific commands.
- **Database Layer**
  Actual databases such as **MySQL, Oracle, SQL Server**, etc., where data is stored.

Working Process

1. The **Java application** sends a request using JDBC API
2. JDBC API forwards the request to **Driver Manager**
3. Driver Manager selects the correct **JDBC driver**
4. The driver communicates with the **database**
5. The result is sent back to the application

Key Points / Properties

- Provides **database independence**
- Supports multiple databases using drivers
- Uses **DriverManager** for connection handling
- Enables execution of **SQL queries**
- Ensures **efficient data retrieval**

---

6. Example (Basic JDBC Code)

```
import java.sql.*;


public class JDBCDemo {

   public static void main(String[] args) {

      try {

         Connection con = DriverManager.getConnection(

            "jdbc:mysql://localhost:3306/test", "root", "password");
```

```
        Statement stmt = con.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM student");

        while (rs.next()) {
            System.out.println(rs.getInt(1) + " " + rs.getString(2));
        }

        con.close();
    } catch (Exception e) {
        System.out.println(e);
    }
  }
}
```

Conclusion

JDBC architecture provides a structured way for Java applications to interact with databases, ensuring flexibility, portability, and efficient data handling across different database systems.

7. Write a Java program to create threads by extending Thread class and using Runnable interface

Definition

A **Thread** in Java is a lightweight process used to achieve **multithreading**, where multiple tasks run **simultaneously**.

Thread Class

The Thread class is used to create a thread by **extending it** and overriding the run() method.

Runnable Interface

Runnable is an interface used to create threads by **implementing run() method**. It is preferred when extending another class.

Syntax

Using Thread Class

```
class MyThread extends Thread {
   public void run() {
      // code
   }
}
```

Using Runnable Interface

```
class MyRunnable implements Runnable {
   public void run() {
      // code
   }
}
```

Program 1: Using Thread Class

```
class T1 extends Thread {
   public void run() {
      try {
         while (true) {
            System.out.println("Good Morning");
            Thread.sleep(1000);
         }
      } catch (Exception e) {}
   }
}

class T2 extends Thread {
   public void run() {
      try {
         while (true) {
            System.out.println("Hello");
            Thread.sleep(2000);
         }
      } catch (Exception e) {}
   }
}

class T3 extends Thread {
   public void run() {
      try {
         while (true) {
```

```
            System.out.println("Welcome");
            Thread.sleep(3000);
        }
    } catch (Exception e) {}
  }
}

public class Main {
  public static void main(String[] args) {
    new T1().start();
    new T2().start();
    new T3().start();
  }
}
```

Program 2: Using Runnable Interface

```
class R1 implements Runnable {
  public void run() {
    try {
        while (true) {
            System.out.println("Good Morning");
            Thread.sleep(1000);
        }
    } catch (Exception e) {}
  }
}

class R2 implements Runnable {
  public void run() {
    try {
        while (true) {
            System.out.println("Hello");
            Thread.sleep(2000);
        }
    } catch (Exception e) {}
  }
}

class R3 implements Runnable {
  public void run() {
    try {
        while (true) {
            System.out.println("Welcome");
            Thread.sleep(3000);
        }
    } catch (Exception e) {}
  }
```

```
}

public class Main2 {
    public static void main(String[] args) {
        new Thread(new R1()).start();
        new Thread(new R2()).start();
        new Thread(new R3()).start();
    }
}
```

Output (Sample)

```
Good Morning
Hello
Welcome
Good Morning
Good Morning
Hello
Good Morning
Welcome
...
```

Output keeps repeating at different intervals.

Key Points

- Thread.sleep(ms) controls execution timing
- Threads run **independently and concurrently**
- Output order may vary
- Runnable is more flexible than Thread

Conclusion

Threads allow multiple tasks to run simultaneously. Both Thread class and Runnable interface are used to create threads, with Runnable being more flexible and widely used.

8. Write the steps involved in setting up the JDBC environment for developing Java database applications

Definition

Setting up the **JDBC environment** means preparing the system so that a Java program can **connect to a database and perform operations** like insert, update, delete, and retrieve data.

Steps to Set Up JDBC Environment

1. Install Java (JDK)

Install JDK and set environment variables (JAVA_HOME, PATH).

2. Install Database Software

Install database like **MySQL / Oracle** and create a database.

3. Download JDBC Driver

Download the required JDBC driver (e.g., MySQL Connector .jar file).

4. Add Driver to Classpath

```
javac -cp .;mysql-connector.jar JDBCDemo.java
java -cp .;mysql-connector.jar JDBCDemo
```

5. Import Required Packages

```
import java.sql.*;
```

6. Load and Register Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

7. Establish Connection

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/test", "root", "password");
```

8. Create Statement

```
Statement stmt = con.createStatement();
```

9. Execute Query

```
ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

10. Process Result

```
while(rs.next()) {
   System.out.println(rs.getInt(1) + " " + rs.getString(2));
}
```

11. Close Connection

```
con.close();
```

Complete Example Program

```
import java.sql.*;

public class JDBCDemo {
   public static void main(String[] args) {
      try {
         // Step 1: Load driver
         Class.forName("com.mysql.cj.jdbc.Driver");

         // Step 2: Establish connection
         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/test", "root", "password");

         // Step 3: Create statement
         Statement stmt = con.createStatement();

         // Step 4: Execute query
         ResultSet rs = stmt.executeQuery("SELECT * FROM student");

         // Step 5: Process result
         while (rs.next()) {
            System.out.println(
               rs.getInt(1) + " " + rs.getString(2));
         }

         // Step 6: Close connection
         con.close();
```

```
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output (Sample)

1 Ravi
2 Anu
3 Kiran

👉 Output depends on database records.

Key Points

• Uses java.sql package
• Requires JDBC driver
• Uses Connection, Statement, ResultSet
• Always close connection

Conclusion

JDBC setup involves installing required tools, configuring drivers, and writing code to connect and interact with databases effectively.

---

9. Discuss in detail about various types of JDBC drivers available in Java with suitable examples

Definition

A **JDBC driver** is a software component that enables Java applications to **communicate with a database**. It acts as a bridge between the **JDBC API and the database**, converting Java calls into database-specific commands.

Explanation

JDBC drivers are classified into **four types** based on how they communicate with the database and how they are implemented.

Types of JDBC Drivers

Type 1: JDBC–ODBC Bridge Driver

• Converts JDBC calls into **ODBC calls**
• Requires ODBC driver installed
• **Deprecated and not portable**

Working:
Java → JDBC → ODBC → Database

Example:

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection con = DriverManager.getConnection("jdbc:odbc:mydsn");
```

Type 2: Native API Driver

• Converts JDBC calls into **native database API calls**
• Requires **database-specific libraries**
• Faster than Type 1 but **platform dependent**

Working:
Java → JDBC → Native API → Database

Example:

```
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con = DriverManager.getConnection(
    "jdbc:oracle:thin:@localhost:1521:xe", "user", "password");
```

Type 3: Network Protocol Driver

• Converts JDBC calls into **database-independent network protocol**
• Uses a **middleware server**
• Platform independent but needs server setup

Working:
Java → JDBC → Middleware → Database

Example:

```
Connection con = DriverManager.getConnection(
    "jdbc:middleware://localhost:5000/db", "user", "password");
```

Type 4: Thin Driver (Pure Java Driver)

• Converts JDBC calls directly into **database-specific protocol**
• Written completely in Java
• **Fastest and most widely used**

Working:
Java → JDBC → Database

Example:

```
Class.forName("com.mysql.cj.jdbc.Driver");
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/test", "root", "password");
```

---

4. Key Points / Properties

• Type 1 → simple but outdated
• Type 2 → faster but platform dependent
• Type 3 → flexible but requires middleware
• Type 4 → fastest and most commonly used
• Type 4 is preferred in modern applications

5. Comparison Table

Type	Name	Speed	Portability	Requirement
Type 1	JDBC–ODBC	Slow	Low	ODBC driver
Type 2	Native API	Medium	Low	Native libraries
Type 3	Network	Medium	High	Middleware server
Type 4	Thin Driver	Fast	High	No extra setup

6. Conclusion

JDBC drivers provide different mechanisms for Java applications to interact with databases. Among them, **Type 4 (Thin Driver)** is the most efficient and widely used due to its high performance and portability

10. Define synchronization in Java. Discuss its importance in multithreaded programming and explain how it is achieved using synchronized methods and blocks.

Synchronization in Java

Definition

Synchronization is a mechanism in Java that controls the access of multiple threads to shared resources, allowing only one thread at a time to execute the critical section, thereby preventing data inconsistency.

Importance in Multithreading

- Prevents **race conditions**
- Maintains **data consistency and integrity**
- Avoids **unexpected or incorrect results**
- Ensures **thread-safe operations**
- Provides **proper coordination between threads**

Critical Section

A **critical section** is the part of the program where shared resources are accessed and needs to be protected.

How Synchronization is Achieved

Java uses **object-level locking (monitor)** to achieve synchronization.

---

1. Synchronized Method

Syntax

```
synchronized void methodName() {
   // critical section
}
```

Explanation

- Entire method is locked
- Only one thread can execute it at a time
- Lock is acquired on the **current object**

Example

```
class Counter {
   int count = 0;

   synchronized void increment() {
      count++;
   }
}

public class Main {
   public static void main(String[] args) throws Exception {

      Counter c = new Counter();

      Thread t1 = new Thread(() -> {
         for(int i=0; i<1000; i++) c.increment();
      });

      Thread t2 = new Thread(() -> {
         for(int i=0; i<1000; i++) c.increment();
      });

      t1.start();
      t2.start();

      t1.join();
      t2.join();

      System.out.println("Final count = " + c.count);
   }
}
```

Output

```
Final count = 2000
```

---

2. Synchronized Block

Syntax

```
synchronized(object) {
   // critical section
}
```

Explanation

- Only specific block is locked
- More flexible than synchronized method
- Lock is applied on a **specified object**

Example

```
class Counter {
   int count = 0;

   void increment() {
      synchronized(this) {
         count++;
      }
   }
}
```

Key Points / Properties

- Ensures **mutual exclusion**
- Uses **object-level locking (monitor)**
- Only one thread enters critical section at a time
- Improves **thread safety**
- Blocks provide **better control over synchronization**

Conclusion

Synchronization is essential in multithreaded programming to ensure safe and consistent access to shared resources. It is implemented using synchronized methods and blocks, which help control thread execution and prevent data conflicts.